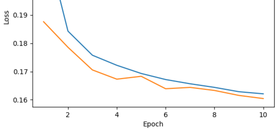
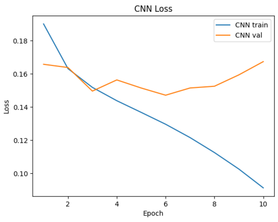
LAB 5

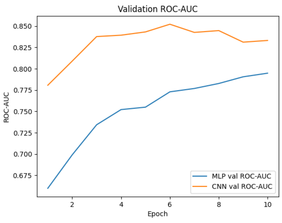
Imports & setup

```
!pip install tensorflow tensorflow-datasets scikit-learn tqdm matplotlib numpy pandas
```

```
Requirement already satisfied: tensorflow in /home/kaggle/usr/lib/google-cloud-sdk/bin:/opt/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/tools/node/bin:/tools/google-cloud-sdk/bin:/opt/google/dotnet:/opt/conda/bin:/usr/local/cuda/bin:/usr/local/nvidia/bin:/usr/local/bin:/usr/bin:/bin:/opt
```







ALL RESULTS FOR ANALYSIS

In [26]:
```
row10 = results_df[results_df["epochs"] == 10].iloc[0]

mlp_params = sum(p.numel() for p in mlp_model.parameters() if p.requires_grad)
cnn_params = sum(p.numel() for p in cnn_model.parameters() if p.requires_grad)

mlp_best_val_auc = float(row10["mlp_best_val_auc"])
cnn_best_val_auc = float(row10["cnn_best_val_auc"])
test_auc_mlp = float(row10["mlp_test_auc"])
test_auc_cnn = float(row10["cnn_test_auc"])
total_time_10 = float(row10["total_train_time_s"])
emissions_kg_10 = float(row10["emissions_kg"])

print("===== EXPERIMENT SUMMARY (10 epochs) =====")
print(f"Device used: {device}")
print(f"Train/Val/Test sizes: {len(train_dataset)}/{len(val_dataset)}/{len(test_dataset)}")
print(f"Number of tags (output dim): {n_tags}")
print("")
print("Model sizes (trainable parameters)")
print(f"MLP params: {mlp_params}")
print(f"CNN params: {cnn_params}")
print("")
print("Validation performance (best across epochs)")
print(f"MLP best val ROC-AUC (10 ep): {mlp_best_val_auc:.4f}")
print(f"CNN best val ROC-AUC (10 ep): {cnn_best_val_auc:.4f}")
print("")
print("Test performance (10 ep models)")
print(f"MLP  | Test ROC-AUC: {test_auc_mlp:.4f}")
print(f"CNN  | Test ROC-AUC: {test_auc_cnn:.4f}")
print("")
print("Training time (approx., 10 ep total MLP+CNN)")
print(f"Total training time: {total_time_10:.1f} s")
print("")
print("Environmental impact (CodeCarbon, 10 ep total)")
print(f"Estimated CO2 emissions (MLP+CNN): {emissions_kg_10:.6f} kg CO2eq")
```

```
===== EXPERIMENT SUMMARY (10 epochs) =====
Device used: mps
Train/Val/Test sizes: 7000/1500/1500
Number of tags (output dim): 50

Model sizes (trainable parameters)
MLP params: 14770
CNN params: 2989810

Validation performance (best across epochs)
MLP best val ROC-AUC (10 ep): 0.7948
CNN best val ROC-AUC (10 ep): 0.8522

Test performance (10 ep models)
MLP  | Test ROC-AUC: 0.7947
CNN  | Test ROC-AUC: 0.8287

Training time (approx., 10 ep total MLP+CNN)
Total training time: 977.9 s

Environmental impact (CodeCarbon, 10 ep total)
Estimated CO2 emissions (MLP+CNN): 0.000520 kg CO2eq
```

EVALUATION WITH BIGGER DATASET BATCH FOR ANALYSIS

In [27]:
```
'''
subset_fraction = 0.5
n_total = len(all_ids)
subset_n = int(n_total * subset_fraction)

rng = np.random.default_rng(123)
subset_idx = rng.choice(n_total, size=subset_n, replace=False)

subset_ids = all_ids[subset_idx]
subset_Y = Y_all_top[subset_idx]

train_ids_half, temp_ids_half, Y_train_half, Y_temp_half = train_test_split(
    subset_ids, subset_Y, test_size=0.3, random_state=42
)

val_ids_half, test_ids_half, Y_val_half, Y_test_half = train_test_split(
    temp_ids_half, Y_temp_half, test_size=0.5, random_state=42
)

train_dataset_half = MagnaTagATuneDataset(
    train_ids_half, Y_train_half, path_map, audio_root, sr_target=16000, duration=3.0
)
val_dataset_half = MagnaTagATuneDataset(
    val_ids_half, Y_val_half, path_map, audio_root, sr_target=16000, duration=3.0
)
test_dataset_half = MagnaTagATuneDataset(
    test_ids_half, Y_test_half, path_map, audio_root, sr_target=16000, duration=3.0
)

train_loader_half = DataLoader(train_dataset_half, batch_size=batch_size, shuffle=True, drop_last=False)
val_loader_half = DataLoader(val_dataset_half, batch_size=batch_size, shuffle=False, drop_last=False)
test_loader_half = DataLoader(test_dataset_half, batch_size=batch_size, shuffle=False, drop_last=False)

mlp_half = MLPTagger(n_mels=n_mels, hidden=128, n_tags=n_tags).to(device)
cnn_half = CNNTagger(n_tags=n_tags, flat_dim=cnn_flat_dim).to(device)

tracker_half = EmissionsTracker(measure_power_secs=10, log_level="error", save_to_file=False)
start_exp_half = time.time()
tracker_half.start()

print("Training MLP model (half dataset)")
history_mlp_half = train_model(
    mlp_half,
    train_loader_half,
    val_loader_half,
    feature_extractor,
    device,
    n_epochs=5,
    lr=1e-3
)

print("Training CNN model (half dataset)")
history_cnn_half = train_model(
    cnn_half,
    train_loader_half,
    val_loader_half,
    feature_extractor,
    device,
    n_epochs=5,
    lr=1e-3
)

emissions_half = tracker_half.stop()
total_time_half = time.time() - start_exp_half

test_loss_mlp_half, test_auc_mlp_half = evaluate(mlp_half, test_loader_half, feature_extractor, device)
test_loss_cnn_half, test_auc_cnn_half = evaluate(cnn_half, test_loader_half, feature_extractor, device)

print("\n====== HALF DATASET SUMMARY =====")
print(f"Subset clips used: {subset_n} de {n_total}")
print(f"Train/Val/Test sizes: {len(train_ids_half)}/{len(val_ids_half)}/{len(test_ids_half)}")
print(f"MLP  | Test loss: {test_loss_mlp_half:.4f} | Test ROC-AUC: {test_auc_mlp_half:.4f}")
print(f"CNN  | Test loss: {test_loss_cnn_half:.4f} | Test ROC-AUC: {test_auc_cnn_half:.4f}")
print(f"Total training time (half dataset): {total_time_half:.1f} s")
print(f"Estimated CO2 emissions (half dataset, MLP+CNN): {emissions_half:.6f} kg CO2eq")


'''
```

Out[27]:
```
'\nsubset_fraction = 0.5\nn_total = len(all_ids)\nsubset_n = int(n_total * subset_fraction)\n\nrng = np.random.default_rng(123)\nsubset_idx = rng.choice(n_total, size=subset_n, replace=False)\n\nsubset_id
s = all_ids[subset_idx]\nsubset_Y = Y_all_top[subset_idx]\n\ntrain_ids_half, temp_ids_half, Y_train_half, Y_temp_half = train_test_split(\n    subset_ids, subset_Y, test_size=0.3, random_state=42\n)\n\nva
l_ids_half, test_ids_half, Y_val_half, Y_test_half = train_test_split(\n    temp_ids_half, Y_temp_half, test_size=0.5, random_state=42\n)\n\ntrain_dataset_half = MagnaTagATuneDataset(\n    train_ids_half,
Y_train_half, path_map, audio_root, sr_target=16000, duration=3.0\n)\nval_dataset_half = MagnaTagATuneDataset(\n    val_ids_half, Y_val_half, path_map, audio_root, sr_target=16000, duration=3.0\n)\ntest_d
ataset_half = MagnaTagATuneDataset(\n    test_ids_half, Y_test_half, path_map, audio_root, sr_target=16000, duration=3.0\n)\n\ntrain_loader_half = DataLoader(train_dataset_half, batch_size=batch_size, shu
ffle=True, drop_last=False)\nval_loader_half = DataLoader(val_dataset_half, batch_size=batch_size, shuffle=False, drop_last=False)\ntest_loader_half = DataLoader(test_dataset_half, batch_size=batch_size,
shuffle=False, drop_last=False)\n\nmlp_half = MLPTagger(n_mels=n_mels, hidden=128, n_tags=n_tags).to(device)\ncnn_half = CNNTagger(n_tags=n_tags, flat_dim=cnn_flat_dim).to(device)\n\ntracker_half = Emissi
onsTracker(measure_power_secs=10, log_level="error", save_to_file=False)\nstart_exp_half = time.time()\ntracker_half.start()\n\nprint("Training MLP model (half dataset)")\nhistory_mlp_half = train_model(\
n    mlp_half,\n    train_loader_half,\n    val_loader_half,\n    feature_extractor,\n    device,\n    n_epochs=5,\n    lr=1e-3\n)\n\nprint("Training CNN model (half dataset)")\nhistory_cnn_half = train_m
odel(\n    cnn_half,\n    train_loader_half,\n    val_loader_half,\n    feature_extractor,\n    device,\n    n_epochs=5,\n    lr=1e-3\n)\n\nemissions_half = tracker_half.stop()\ntotal_time_half = time.tim
e() - start_exp_half\n\ntest_loss_mlp_half, test_auc_mlp_half = evaluate(mlp_half, test_loader_half, feature_extractor, device)\ntest_loss_cnn_half, test_auc_cnn_half = evaluate(cnn_half, test_loader_hal
f, feature_extractor, device)\n\nprint("\n====== HALF DATASET SUMMARY =====")\nprint(f"Subset clips used: {subset_n} de {n_total}")\nprint(f"Train/Val/Test sizes: {len(train_ids_half)}/{len(val_ids_hal
f)}/{len(test_ids_half)}")\nprint(f"MLP  | Test loss: {test_loss_mlp_half:.4f} | Test ROC-AUC: {test_auc_mlp_half:.4f}")\nprint(f"CNN  | Test loss: {test_loss_cnn_half:.4f} | Test ROC-AUC: {test_auc_cnn_h
alf:.4f}")\nprint(f"Total training time (half dataset): {total_time_half:.1f} s")\nprint(f"Estimated CO2 emissions (half dataset, MLP+CNN): {emissions_half:.6f} kg CO2eq")\n\n\n\n'
```

SUMMARY FOR REPORT

In [28]:
```
results_df.to_csv("lab5_results_epochs.csv", index=False)

row10 = results_df[results_df["epochs"] == 10].iloc[0]

summary_df = pd.DataFrame({{
    "device": str(device),
    "train_size": len(train_dataset),
    "val_size": len(val_dataset),
    "test_size": len(test_dataset),
    "n_tags": n_tags,
    "mlp_params": mlp_params,
    "cnn_params": cnn_params,
    "mlp_best_val_auc": float(row10["mlp_best_val_auc"]),
    "cnn_best_val_auc": float(row10["cnn_best_val_auc"]),
    "mlp_test_auc": float(row10["mlp_test_auc"]),
    "cnn_test_auc": float(row10["cnn_test_auc"]),
    "total_time_10": float(row10["total_train_time_s"]),
    "emissions_kg_10": float(row10["emissions_kg"]),
}})

summary_df.to_csv("lab5_summary_final.csv", index=False)
```

Save as HTML

In [4]:
```
import os

os.system('jupyter nbconvert --to html yourNotebook.ipynb')

!jupyter nbconvert --to html Manuel-Castillo-lab5.ipynb
```

```
usage: jupyter [-h] [--version] [--config-dir] [--data-dir] [--runtime-dir]
               [--paths] [--json] [--debug]
               [subcommand]

Jupyter: Interactive Computing

positional arguments:
  subcommand      the subcommand to launch

options:
  -h, --help      show this help message and exit
  --version       show the versions of core jupyter packages and exit
  --config-dir    show Jupyter config dir
  --data-dir      show Jupyter data dir
  --runtime-dir   show Jupyter runtime dir
  --paths         show all Jupyter paths. Add --json for machine-readable
                  format.
  --json          output paths as machine-readable json
  --debug         output debug information about paths

Available subcommands: kernel kernelspec migrate run troubleshoot

Jupyter command `jupyter-nbconvert` not found.
usage: jupyter [-h] [--version] [--config-dir] [--data-dir] [--runtime-dir]
               [--paths] [--json] [--debug]
               [subcommand]

Jupyter: Interactive Computing

positional arguments:
  subcommand      the subcommand to launch

options:
  -h, --help      show this help message and exit
  --version       show the versions of core jupyter packages and exit
  --config-dir    show Jupyter config dir
  --data-dir      show Jupyter data dir
  --runtime-dir   show Jupyter runtime dir
  --paths         show all Jupyter paths. Add --json for machine-readable
                  format.
  --json          output paths as machine-readable json
  --debug         output debug information about paths

Available subcommands: kernel kernelspec migrate run troubleshoot

Jupyter command `jupyter-nbconvert` not found.
```